

Sprint 01 Operational Action Plan: Parallel Engineering & Baseline Demo

Sprint Window: October 1, 2026 – October 7, 2026 · **Target Deadline:** Tuesday Faculty Progress Review
Official GitHub Repository: <https://github.com/Chaitanya-666/agriAgent>

Sprint 01 Primary Objective & Non-Blocking Design

The Core Challenge: As of October 1, the full datasets are not yet staged, and model specializations are in progress. If the team works sequentially (e.g., UI waiting for AI models, AI waiting for finalized datasets), **zero code will be written this week.**

The Solution — Parallel Decoupling: Every member has a completely isolated technical domain with strict input/output contracts. Amit builds the UI using mock visual overlays; Sahil pulls and stages the cotton datasets independently; Sumant ingests ICAR agronomy guides into ChromaDB independently; and Chaitanya wraps the zero-shot vision foundation models on sample images. By Sunday night, each member opens a clean Pull Request to main, assembling our working baseline for Tuesday's faculty demo!

1. Granular Member Task Workflows (Sprint 01)

Track 1: Core AI & Vision Systems — Chaitanya Shinde

Assigned Lead: Chaitanya Shinde

Target Branch: chaitanya/foundation-vision

Working Directories: src/vision/, src/state.py, scripts/

Technical Context: The zero-shot capabilities of Grounding DINO and SAM 2 allow initial development to proceed immediately on **any** sample agricultural images, completely unblocked by dataset downloads.

1. Task 1.1 (Repository Integrity & State Finalization):

- Verify that src/state.py cleanly exports Task, Artifact, ArtifactType, and merge_dicts alongside the vision schema (Box, Mask, GraphState).
- Run AST and import sanity tests to ensure zero circular dependencies.

2. Task 1.2 (Grounding DINO-T Zero-Shot Wrapper):

- Create src/vision/grounding_dino.py:
 - Load IDEA-Research/grounding-dino-tiny from HuggingFace Transformers.
 - Implement function predict_boxes(image: PIL.Image, prompt: str, box_thresh: float = 0.3, text_thresh: float = 0.25) -> list[Box].
 - Support FP16 execution when CUDA is detected, falling back smoothly to CPU.

3. Task 1.3 (SAM 2-Tiny Instance Segmentation Wrapper):

- Create src/vision/sam2_wrapper.py:
 - Load facebook/sam2-hiera-tiny using SAM2ImagePredictor.

- Implement function `predict_masks(image: PIL.Image, boxes: list[Box]) -> list[Mask]`.
- Ingest bounding boxes and return 2D binary numpy arrays with predicted IoU scores.

4. Task 1.4 (Refinement Loop Integration):

- Connect `src/vision/refinement.py` to the predicted masks: if predicted IoU < 0.85, compute the error centroid (x_c, y_c) and re-query SAM 2 with the corrective point prompt.

5. Sprint 01 Deliverable (Sunday):

- Runnable script `scripts/test_vision_pipeline.py` that takes a field photo, executes detection and segmentation, and outputs boxes and masks without errors.

—

Track 2: Multimodal VLM & Agronomy Intelligence — Sumant Vetal

Assigned Lead: Sumant Vetal

Target Branch: `sumant/vlm-agronomy`

Working Directories: `data/knowledge_base/`, `src/agents/agronomy_agent.py`

Technical Context: Sumant builds the intelligence layer that translates detected weed and lesion masks into actionable agricultural advice and chemical prescriptions for farmers.

1. Task 2.1 (ICAR & CIBRC Agronomy Manual Ingestion):

- Download 3-5 official agricultural extension bulletins:
 - ICAR-CICR (Central Institute for Cotton Research, Nagpur) Cotton Weed Management Guide.
 - CIBRC approved list of cotton herbicides (Glyphosate 41% SL, Pendimethalin, Quizalofop-ethyl).
 - Cotton foliar disease compendium (Bacterial Blight, Alternaria Leaf Spot, Cotton Leaf Curl Virus).
- Convert or place plain text / markdown / PDF versions into `data/knowledge_base/`.

2. Task 2.2 (ChromaDB Vector Store Setup):

- Create `src/agents/agronomy_rag.py`:
 - Ingest the knowledge base documents into a local ChromaDB collection using sentence-transformer embeddings (all-MiniLM-L6-v2).
 - Implement semantic search function: `retrieve_agronomy_context(query: str, top_k: int = 3) -> list[str]`.

3. Task 2.3 (Multimodal Visual Reasoning with Qwen3-VL / InternVL 3.5 & Groq):

- Evaluate modern open-weight VLMs in `notebooks/01_vlm_reasoning_experiment.ipynb` (Qwen3-VL-2B/4B or InternVL3.5-4B) for visual lesion and weed symptom reasoning on sample images.
- Create a prescription generator script using Groq Free Tier (llama-3.3-70b-versatile):
 - Prompt: Ingest detected weed/disease name + retrieved ICAR guidelines.
 - Output: Structured JSON containing recommended chemical trade name, dilution ratio (ml per liter of water), safety buffer, and targeted spot-spray instructions.

4. Sprint 01 Deliverable (Sunday):

- Runnable script `scripts/test_agronomy_rag.py` and evaluation notebook proving that querying “Parthenium infestation in cotton” returns the exact ICAR chemical dosage, VLM visual reasoning, and treatment timeline.

—

Track 3: Data Science & Empirical Benchmarking — Sahil Chavan

Assigned Lead: Sahil Chavan

Target Branch: `sahil/cotton-datasets`

Working Directories: `data/datasets/`, `src/evaluation/`, `scripts/`

Technical Context: Sahil owns the empirical evidence. His objective is to acquire the cotton datasets, prepare a clean 50-image test set, and verify our evaluation formulas.

1. Task 3.1 (Dataset Acquisition & Staging):

- Setup Kaggle API on local machine or Google Drive (kaggle datasets download).
- Download the primary Indian cotton weed benchmarks:
 - CottonWeeds (Kaggle: 7,578 Indian field photos).
 - Cotton-Weed-12-Class (5,648 images with 9,000+ bounding box annotations).
 - Cotton Plant Disease & Pest Dataset (leaf lesions: Aphids, Armyworm, Blight).
- Download a lightweight 485 MB sample of Sugar Beets 2016 (from DatasetNinja).

2. Task 3.2 (50-Image Benchmark Test Split):

- Filter and curate data/datasets/test_cotton_50/:
 - 30 high-contrast images of cotton crops with visible weed competition.
 - 20 leaf photos showing identifiable disease spots or pest damage.
- Stage corresponding ground-truth annotations (bounding boxes and masks).

3. Task 3.3 (Evaluation Script Standardization):

- Review and execute [src/evaluation/iou_dice.py](../src/evaluation/iou_dice.py):
 - Verify compute_iou and compute_dice against dummy binary arrays.
 - Verify compute_herbicide_savings dilation logic ($1 - \frac{\text{weed} + \text{buffer}}{\text{total}}$).
- Create src/evaluation/data_loader.py to standardize loading raw images and masks into NumPy arrays.

4. Sprint 01 Deliverable (Sunday):

- The directory data/datasets/test_cotton_50/ staged with verified samples, and a verification script scripts/verify_datasets.py that loads an image, prints its dimensions, and checks label alignment.

—

Track 4: Geospatial Systems & Full-Stack Deployment — Amit Ingle

Assigned Lead: Amit Ingle

Target Branch: amit/streamlit-ui

Working Directories: app/, src/vision/visualization.py

Technical Context: Amit builds the visual application that will be projected in our faculty meetings. He does not need live model weights to build a responsive, complete UI.

1. Task 4.1 (Production Streamlit Scaffold):

- Build app/app.py using Streamlit (pip install streamlit):
 - Branded institutional header: **“AgriAgent: Dual-Brain Multi-Agent Precision Agriculture (VJTI FYP)”**.
 - Sidebar controls:
 - Application Dropdown: Site-Specific Weed Spraying, Disease Auditing, Stand Count, Chlorosis.
 - Detection Confidence Slider (0.1 to 0.9, default 0.3).
 - Spray Nozzle Buffer Slider (5cm to 20cm, default 10cm).
 - Drag-and-drop file uploader accepting .jpg, .jpeg, .png.

2. Task 4.2 (Real-Time Mask Overlay Compositor):

- Create src/vision/visualization.py:
 - Implement overlay_masks(image, masks, alpha=0.45) -> PIL.Image: Uses OpenCV cv2.addWeighted to paint semi-transparent colored masks (Green = Crop, Red = Weed, Yellow = Disease).
 - Implement draw_boxes(image, boxes) -> PIL.Image: Renders bounding box outlines with class tags and confidence percentages.

3. Task 4.3 (Dual-Panel Spray Map & Metrics Dashboard):

- Main screen layout in app/app.py:

- ▶ Two responsive columns: **Original Field Photo** vs. **Precision Spray Map**.
 - Dynamic Agronomic KPI Cards (Bottom):
 - ▶ **Weed Infestation Area: 18.2%**
 - ▶ **Selective Chemical Reduction: 74.6% Saved** vs broadcast spraying
 - ▶ **Actionable Status: “Selective Spot-Spray Prescription Generated”**
 - Download Button: Export prescription summary as CSV or GeoJSON mock.
4. **Sprint 01 Deliverable (Sunday):**
- Fully interactive Streamlit dashboard (`streamlit run app/app.py`) running on localhost, demonstrating file upload, threshold adjustment, and simulated mask compositing.

2. GitHub Collaboration & Branch Merging Protocol

To ensure that the `main` branch remains stable, deployable, and protected against regressions, all four team members must strictly adhere to the following git workflow:

Branch Protection Rules Enforcement

1. **Direct pushes to main are blocked:** Any attempt to run `git push origin main` will be rejected by GitHub's server-side ruleset.
2. **Named Feature Branches Only:** Every member works strictly within their designated branch.
3. **Zero Large File Commits:** Never run `git add .` blindly. Heavy dataset zip files (> 50MB) and `.venv` directories are gitignored and must never enter commit history.

2.1 Daily Step-by-Step Git Commands

```
# Step 1: Clone the repository (first time only)
git clone https://github.com/Chaitanya-666/agriAgent.git
cd agriAgent

# Step 2: Create and switch to your designated personal branch
# For Chaitanya: git checkout -b chaitanya/foundation-vision
# For Sumant:    git checkout -b sumant/vlm-agronomy
# For Sahil:     git checkout -b sahil/cotton-datasets
# For Amit:      git checkout -b amit/streamlit-ui
git checkout -b <your-name>/<feature-name>

# Step 3: Implement your tasks in your designated folder...

# Step 4: Check modified files and stage ONLY your code
git status
git add <path-to-your-file>
# Example: git add app/app.py src/vision/visualization.py

# Step 5: Commit with a meaningful conventional commit message
git commit -m "feat(ui): implement dual-panel streamlit layout and mask overlay"

# Step 6: Push your branch to GitHub
git push -u origin <your-name>/<feature-name>
```

2.2 Pull Request & Sunday Merging Schedule

- **Sunday 6:00 PM IST (Code Freeze):** All four members push their final commits and open a Pull Request (PR) on GitHub targeting `main`.

- **Sunday 8:00 PM IST (Integration & Merge Review):** Project Lead Chaitanya Shinde reviews PR diffs, executes syntax checks, and merges accepted PRs into main.
- **Monday Morning:** All members pull the integrated main branch to their local setups:

```
git checkout main
git pull origin main
```
- **Tuesday Morning:** Final rehearsal of the unified main branch before the progress presentation with Prof. V. D. Dhore.

3. Sprint 01 Deliverables & Checklist Summary

Member	Designated Branch	Sunday Tangible Deliverable	Status Gate
Chaitanya Shinde	chaitanya/vision	Python wrappers for Grounding DINO + SAM 2 (scripts/test_vision_pipeline.py)	Mandatory
Sumant Vetal	sumant/agronomy	ChromaDB Agronomy RAG populated with ICAR guides (scripts/test_agronomy_rag.py)	Mandatory
Sahil Chavan	sahil/datasets	50-image Cotton benchmark test set staged (data/datasets/test_cotton_50/)	Mandatory
Amit Ingle	amit/ui	Interactive Streamlit dashboard running on localhost (app/app.py)	Mandatory

AgriAgent Sprint 01 Operational Specification · Department of Computer Engineering, VJTI Mumbai.
 All code subject to peer review prior to main branch integration.